Python I. beadandó feladat

ELTE IK PNYFT – 2026 D feladat

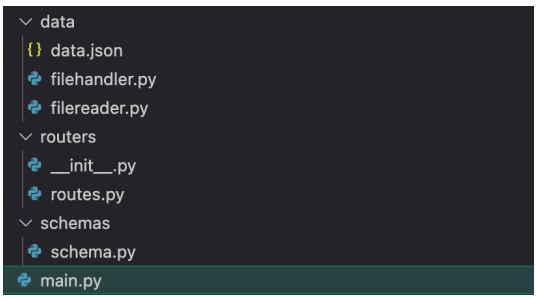
REST API készítése FastAPI alapokon – Éttermi asztal- és rendeléskezelő

A beadandó feladatban elkészítendő API egy étterem (a Pythonia Bisztró) vendégeit és rendeléseit kezeli. Lehetővé teszi a vendégek felvételét, egy asztal megnyitását (számla nyitását) egy adott vendégnek, valamint ételek és italok rendelését a vendég számlájára az API végpontok segítségével.

A rendelt tételek módosíthatók (pl. az adagszám), vagy le is mondhatók. Ha egy vendég ugyanazt az ételt újra megrendeli, a meglévő tétel adagszáma nő, nem jön létre új tétel. Egy asztalnál egyszerre csak egy nyitott számla lehet. Le lehet kérdezni egy vendég fizetendő végösszegét, amely tartalmazza a 12%-os szervizdíjat, és opcionálisan kedvezményt is tartalmazhat. Az API ezeken kívül lehetővé teszi egy adott vendég adatainak lekérdezését, az összes vendég megjelenítését és egy vendég számláján lévő tételek listázását. Ezekkel a funkciókkal elkészíthető egy éttermi kiszolgáló alkalmazás felülete is. A felhasználói felületet nem kell elkészíteni, de ha az API-hoz készül ilyen felület, az extra pontot érhet.

Az API a végpontok által kezelt adatokat a data.json nevű fájlban tárolja. A fájl tartalma az erre a célra kialakított végpontokkal menthető és visszatölthető. A feladat tehát egy FastAPI implementáció kiegészítése. A kód egy része már meg van írva a mellékelt könyvtárakban, az előre elkészített fájlokban pedig instrukciók segítik az adott modul funkcióinak megírását és véglegesítését.

A mellékelt fájlok tartalma



- A data.json az adatok tárolására szolgál JSON formátumban. Ebben elhelyeztünk néhány tesztadatot (3 vendég, 3 számla), amelyekkel az alkalmazás kipróbálható. A fájl két listát tartalmaz: `Guests` és `Tabs`.
- A filehandler.py és a filereader.py a JSON adatok fájlba írására, illetve fájlból történő beolvasására használható. A függvények fejlécei és a használatukat bemutató példák már szerepelnek a fájlokban, a függvények törzsét kell megírni.

- A routes.py tartalmazza az alkalmazás végpontjait. Ezeket a függvényeket kell kiegészíteni a megfelelő funkciókkal és paraméterekkel.
- A schema.py a JSON adatok kliens és szerver közötti mozgatására, valamint kezelésére alkalmas osztályokat tartalmazza. Ezekben a mezők már fel vannak sorolva, de a validáció és az étel-kategóriák felsorolása (Enum) még hiányzik. Az osztályokat a fájlban leírtak alapján ki kell egészíteni, ügyelve az adatok helyességére.
- A main.py tartalmazza a FastAPI main modulját, amelyet a uvicorn Python csomag segítségével kell futtatni az alkalmazás elindításához.

Az adatok szerkezete

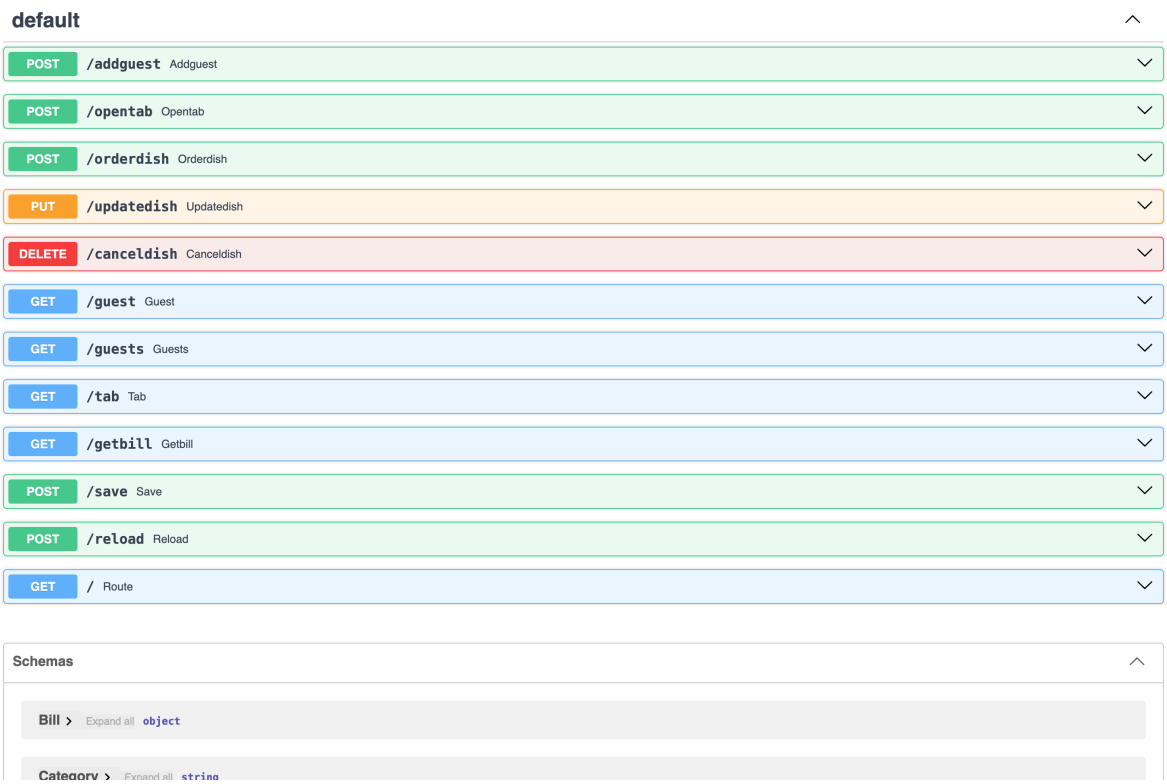
Egy számla rekordja a data.json fájlban (a rendelt ételekkel együtt):

```
{
  "id": 101,
  "guest_id": 1,
  "table_no": 4,
  "dishes": [
    {"dish_id": 201, "name": "Gulyásleves", "category": "leves", "price":
2490.0, "portions": 2},
    {"dish_id": 203, "name": "Ásványvíz", "category": "ital", "price":
690.0, "portions": 3}
  ]
}
```

Az árak forintban értendők, egy adag árát jelentik. A kategória csak a következők egyike lehet: leves, előétel, főétel, desszert, ital. Minden vendégnek legfeljebb egy nyitott számlája lehet.

Feladatok

Az itt látható végpontokat kell elkészítenünk:



- **Az addguest segítségével egy vendéget tudunk felvenni a rendszerbe. (5 pont)**
- **Az opentab egy asztalt nyit (számlát rendel) egy vendéghez. (5 pont)**
- **Az orderdish egy ételt rendel az adott vendég számlájára. (5 pont)**
- **Az updatedish módosítja egy rendelt étel adatait a vendég számláján. (10 pont)**
- **A canceldish lemond (töröl) egy ételt a számláról. (5 pont)**
- **A guest egy adott vendég adatait adja vissza. (5 pont)**
- **A guests visszaadja az összes vendéget. (5 pont)**
- **A tab egy vendég számláján lévő összes étel adatait adja vissza. (5 pont)**
- **A getbill egy vendég fizetendő végösszegét adja vissza szervizdíjjal és kedvezménnyel. (10 pont)**
- **A save és a reload az adatok mentésére és visszatöltésére szolgál; a visszatöltés a meglévő adatok cseréjével vagy összefésülésével is történhet. (5 + 10 pont)**
- A Route (/) az étterem nevét adja vissza. Ez a funkció már implementálva van a main.py fájlban.

Az endpoint-ok teszteléséhez indítsuk el a REST API-t, és nézzük meg a listát a FastAPI által automatikusan generált webes felületen. Az API elindításakor a rendszer kiírja azt az URL-t és portot, amelyen a program fut. Ezt az URL-t a /docs végződéssel kiegészítve érjük el a webes felületet. Az alkalmazás elindítása:

```
uvicorn mainfileneve:appneve --reload
```

Az alkalmazás webes felületén (/docs) a fenti feladatok mind elvégezhetők. Minden funkció a routes.py fájlban és ebben a leírásban meghatározott módon viselkedik, és az ott leírt

adatokat adja vissza szabványos JSON formátumban, a JSONResponse függvénnyel, státuszkóddal együtt.

A végpontok megírása során az alábbi szabályokat kell betartani (10 pont):

- Minden végpontnál adjuk meg a response_model értékét (típus).
- Ügyeljünk a típusok megadására a függvények paramétereinél és visszatérési értékénél is.
- Ahol lehet, a függvények visszatérési értéke JSONResponse() legyen.
- Minden függvény tartalmazzon hibakezelést: hiba esetén dobjon HTTPException-t a megfelelő státuszkóddal.
- A függvények a JSON adatok mentéséhez és visszaolvasásához a filehandler.py és a filereader.py fájlt használják. Tegyük elérhetővé ezeket az alkalmazásban! Az adatokat a data.json fájlba kell menteni.
- A HTTP válaszok minden esetben tartalmazzák a megfelelő státuszkódot (lásd a következő fejezet táblázatát), és a végpontok a megfelelő HTTP metódust használják.

Az alkalmazás használata

A main.py fájl a REST API main modulja, ez indítja el az alkalmazást. A futtatásához telepíteni kell a pip csomagkezelővel a uvicorn, a fastapi és az e-mail cím validálásához szükséges pydantic[email] csomagokat:

```
pip install uvicorn fastapi "pydantic[email]"
```

vagy

```
pip3 install uvicorn fastapi "pydantic[email]"
```

Ezután az alkalmazás az API könyvtárában állva a következő paranccsal futtatható a terminálban:

```
uvicorn main:app --reload --port 9000
```

A port nem kötelező opció. A futó alkalmazás a következő URL-en érhető el:

```
http://127.0.0.1:9000
```

A webes UI (Swagger) pedig az alábbi URL használatával:

```
http://127.0.0.1:9000/docs
```

A dokumentáció pedig itt:

```
http://127.0.0.1:9000/redoc
```

A HTTP protokoll helyes használata

A REST API-ban a HTTP metódus fejezi ki, hogy mit csinál a végpont, a státuszkód pedig azt, hogy mi lett a kérés eredménye. A kérések és válaszok törzse mindig JSON formátumú (Content-Type: application/json). A végpontok megírásakor az alábbi táblázatokat kell követni.

HTTP metódus	Mire használjuk	Példa ebben a feladatban
GET	Adat lekérdezése, nem módosít semmit	/guest, /guests, /tab, /getbill
POST	Új adat létrehozása	/addguest, /opentab, /orderdish
PUT	Meglévő adat módosítása (teljes csere)	/updatedish
DELETE	Adat törlése	/canceldish

Státuszkód	Jelentése	Mikor használjuk
200 OK	Sikeres kérés	Sikeres lekérdezés, módosítás, törlés, visszatöltés
201 Created	Létrejött az új erőforrás	Sikeres létrehozás (POST), mentés
400 Bad Request	Hibás kérés (üzleti szabály sérül)	pl. azonosító eltérés, limit túllépése, hibás fájlnév
404 Not Found	Nem található	Nem létező azonosító, fájl
409 Conflict	Ütközés a meglévő adatokkal	Már létező azonosító, már létező rekord, ellentmondásos adatok visszatöltéskor
422 Unprocessable Entity	A bemenő adat nem felel meg a sémának	Pydantic validációs hiba (ezt a FastAPI automatikusan adja)

Hibás kérés esetén a végpont **HTTPException**-t dobjon a megfelelő státuszkóddal és egy rövid, magyar nyelvű hibaüzenettel, pl.:

```
raise HTTPException(status_code=404, detail="Nem található hallgató a
megadott ID-val: 99")
```

Opcionális: futtatás HTTPS protokollon (extra pont)

A uvicorn képes titkosított (HTTPS) kapcsolaton is kiszolgálni az API-t, ehhez egy tanúsítvány és egy hozzá tartozó kulcs kell. Fejlesztéshez elegendő egy saját aláírású (self-signed) tanúsítvány, amelyet az OpenSSL programmal lehet elkészíteni (macOS-en és Linuxon alapból telepítve van, Windowson a Git for Windows tartalmazza):

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -
days 365 -subj "/CN=localhost"
uvicorn main:app --reload --port 9443 --ssl-keyfile key.pem --ssl-certfile
cert.pem
```

Ezután az API a https://127.0.0.1:9443/docs címen érhető el. A böngésző figyelmeztetni fog, hogy a tanúsítvány nem megbízható. Ez saját aláírású tanúsítványnál normális, a

figyelmeztetést el lehet fogadni. A Postmanben ilyenkor a Settings → SSL certificate verification beállítást kell kikapcsolni. A key.pem fájlt nem szabad beadni vagy megosztani!

Az adatok validálása (15 pont)

Az adatok kezelésére használható osztályokat a megadott séma alapján ki kell dolgozni. A schema.py tartalmazza az adatok küldésére és fogadására készített osztályokat. Az osztályokban az adatok legyenek validálva:

- Az int típusú azonosítók (id, guest_id, dish_id) nem lehetnek negatívak.
- Az email mező csak e-mail formátumot fogadhat el (EmailStr).
- A telefonszám formátuma: +36, utána 9 számjegy (pl. +36301234567). A szóközöket és kötőjeleket a validálás előtt el kell távolítani, így a `+36 30 123 4567` is elfogadott, és `+36301234567` alakban kerül tárolásra.
- Az étel neve nem lehet üres, az ára pozitív szám, az adagszám 1 és 20 közötti egész szám.
- Az étel kategóriája csak a Category felsorolás (Enum) egyik értéke lehet: leves, előétel, főétel, desszert, ital.
- Az asztalszám 1 és 30 közötti egész szám.
- Modell szintű validáció a Tab osztályban: egy számlán egy dish_id csak egyszer szerepelhet.
- Modell szintű validáció a DataStore osztályban (a teljes data.json): egyedi vendég- és számla-azonosítók, egy vendégnek legfeljebb egy számlája lehet, egy asztalhoz legfeljebb egy számla tartozhat, és minden számla létező vendéghez tartozik. Ezt a mentés és a visszatöltés is használja.
- Hiba esetén ValueErrort kell dobni, és lehetőség szerint a kliens oldalon is jelezni kell a hibát. A FastAPI a modellből eredő hibát automatikusan 422-es válasszal jelzi.

Segítség a validáláshoz

Mezőszintű validáció (`field_validator`): a dekorált osztálymetódus megkapja a mező értékét. Hibás érték esetén ValueErrort dob, egyébként visszaadja az (esetleg átalakított) értéket. Szöveges minták ellenőrzésére a Python beépített `re` (reguláris kifejezés) modulja használható: a `re.fullmatch(minta, szöveg)` akkor ad vissza nem-None értéket, ha a teljes szöveg illeszkedik a mintára.

```
import re
from pydantic import BaseModel, field_validator

class Csomag(BaseModel):
    irsz: str                      # pl. "1117"

    @field_validator("irsz")
    @classmethod
    def validate_irsz(cls, value: str) -> str:
        value = value.replace(" ", "")          # átalakítás is lehetséges
        if not re.fullmatch(r"\d{4}", value):    # \d{4}: pontosan 4
számjegy
            raise ValueError("Az irányítószám 4 számjegyből áll")
        return value
```

Hasznos mintaelemek: `\+` a + karakter (a + önmagában ismétlést jelent, ezért kell elé a \), `\d` számjegy, `{9}` pontosan 9 ismétlés.

Felsorolás típus (`Enum`): ha egy mező csak előre megadott értékeket vehet fel, a Python beépített `enum` moduljával felsorolás típust készíthetünk. Ha az osztály a `str`-ből és az `Enum`-ból is öröklődik, az értékei szövegként jelennek meg a JSON-ban, a Pydantic pedig automatikusan 422-es hibát ad minden más értékre. A /docs felületen a lehetséges értékek legördülő listában is megjelennek.

```
from enum import Enum

class Color(str, Enum):
    RED = "piros"
    GREEN = "zöld"

class Car(BaseModel):
    color: Color          # csak "piros" vagy "zöld" lehet

car = Car(color="piros")
car.model_dump()              # {'color': <Color.RED: 'piros'>}  ->
json.dump nem tudja kiírni!
car.model_dump(mode="json")   # {'color': 'piros'}               -> így kell
fájlba írni / JSONResponse-ba tenni
```

Modell szintű validáció (`model_validator`): akkor kell, ha a szabály több mezőt (vagy egy lista összes elemét) érinti. Az `after` módú validátor a mezők egyenkénti ellenőrzése után fut, és a már elkészült objektumot (self) kapja meg:

```
from pydantic import model_validator

class Rendeles(BaseModel):
    tetelek: List[Tetel]

    @model_validator(mode="after")
    def validate_tetelek(self) -> "Rendeles":
        if len(self.tetelek) > 10:
            raise ValueError("Legfeljebb 10 tétel lehet")
        return self
```

Validálás a végpontban: ha egy már tárolt (dict) adatot szeretnénk módosítás előtt ellenőrizni, a modell `model_validate()` metódusa használható. Hiba esetén `pydantic.ValidationError` kivétel keletkezik, amelyet el kell kapni, és HTTPException-né kell alakítani. Ez például az orderdish végpontban hasznos, amikor az összeadott adagszám túllépné a 20-at:

```
from pydantic import ValidationError

try:
    merged = Dish.model_validate({**existing, "portions":
existing["portions"] + dish.portions})
except ValidationError as e:
    raise HTTPException(status_code=400, detail=e.errors()[0]["msg"])
```

Query paraméter validálása (`Query`): egyszerű (nem modell típusú) paraméterekre is megadható korlát a FastAPI `Query` függvényével. Érvénytelen érték esetén a FastAPI automatikusan 422-es választ ad:

```
from fastapi import Query

@routers.get('/pelda')
def pelda(limit: int = Query(10, ge=1, le=100)):   # alapérték 10, 1 és 100
között
    ...
```

A végpontok implementációja

Az alábbi végpontokat kell kidolgozni. A paraméterek neve és típusa a megadott legyen. Az egyszerű paraméterek query paraméterek (pl. `/guest?guestid=1`), a modell típusú paraméterek a kérés JSON törzsében érkeznek.

Vendég felvétele. A bemenet egy Guest típus, a visszatérési érték a felvitt vendég rekordja (201). Ha az azonosító már foglalt: 409.

```
@routers.post('/addguest', response_model=Guest, status_code=201)
def addguest(guest: Guest) -> Guest:
    pass
```

Asztal nyitása (számla hozzárendelése) egy vendéghez. A bemenő paraméterek a vendég azonosítója és az asztal száma. Az új számla azonosítója a legnagyobb meglévő számla-azonosító + 1, az ételek listája kezdetben üres. A visszatérési érték a következő szöveg: „Sikeres asztalnyitás.” (201). Nem létező vendég: 404, ha a vendégnek már van számlája, vagy az asztalnál már ül valaki (van hozzá számla): 409, érvénytelen asztalszám: 422.

```
@routers.post('/opentab', response_model=str, status_code=201)
def opentab(guestid: int, tableno: int) -> str:
    pass
```

Étel rendelése a vendég számlájára. A bemenő paraméterek a vendég azonosítója és az étel. A visszatérési érték a teljes számla. Új étel esetén 201 a státuszkód. Ha ugyanaz az étel (azonos dish_id, név, kategória és ár) már szerepel a számlán, az adagszámok összeadódnak, és 200 a válasz. Ha az összeadott adagszám 20 fölé menne: 400. Ha a dish_id már egy másik ételhez tartozik: 409. Nincs számla: 404.

```
@routers.post('/orderdish', response_model=Tab, status_code=201)
def orderdish(guestid: int, dish: Dish) -> Tab:
    pass
```

Példa kérés: `POST /orderdish?guestid=3`, a kérés törzse:

```
{"dish_id": 301, "name": "Halászlé", "category": "leves", "price": 3290.0,
"portions": 1}
```

Egy rendelt étel adatainak módosítása. A bemenő paraméterek a vendég azonosítója, az étel azonosítója, valamint az étel új adatai. Az étel cseréjével is megoldható a feladat, de az étel azonosítója nem változhat: ha a törzsben lévő dish_id eltér a dishid paramétertől, 400 a válasz. Nincs számla, vagy nincs ilyen étel a számlán: 404. A visszatérési érték a teljes számla (200).

```
@routers.put('/updatedish', response_model=Tab)
def updatedish(guestid: int, dishid: int, dish: Dish) -> Tab:
    pass
```

Egy étel lemondása (törlése) a vendég számlájáról. A bemenő paraméterek a vendég azonosítója és az étel azonosítója. A visszatérési érték a teljes számla (200). Nincs számla vagy nincs ilyen étel: 404.

```
@routers.delete('/canceldish', response_model=Tab)
def canceldish(guestid: int, dishid: int) -> Tab:
    pass
```

Egy adott vendég adatainak megjelenítése. A bemenő paraméter a vendég azonosítója, a visszatérési érték a vendég rekordja (200). Nem létező vendég: 404.

```
@routers.get('/guest', response_model=Guest)
def guest(guestid: int) -> Guest:
    pass
```

Az összes vendég lekérdezése. Nincs bemenő paraméter, a számlák nem kerülnek megjelenítésre. A visszatérési érték a vendégek listája (200).

```
@routers.get('/guests', response_model=list[Guest])
def guests() -> list[Guest]:
    pass
```

Egy adott vendég számláján lévő ételek megjelenítése. A paramétere a vendég azonosítója, a visszatérési érték az ételek listája (200). Üres számla esetén üres lista, ha nincs számla: 404.

```
@routers.get('/tab', response_model=list[Dish])
def tab(guestid: int) -> list[Dish]:
    pass
```

Egy adott vendég fizetendő végösszege. A bemenő paraméterek a vendég azonosítója és az opcionális kedvezmény százalékban (alapértéke 0, megengedett értéke 0 és 50 között van, ezen kívül 422). A visszatérési érték egy Bill objektum (200). Ha nincs számla: 404. A számítás szabályai (minden összeg két tizedesjegyre kerekítve):

- `subtotal`: Σ(ár × adag) a számlán lévő ételekre,
- `service_charge`: a subtotal 12%-a,
- `discount_amount`: (subtotal + service_charge) × kedvezmény / 100,
- `total`: subtotal + service_charge – discount_amount.

```
@routers.get('/getbill', response_model=Bill)
def getbill(guestid: int, discount: int = Query(0, ge=0,
le=MAX_DISCOUNT_PERCENT)) -> Bill:
    pass
```

Példa: a mellékelt data.json alapján a `GET /getbill?guestid=1&discount=10` válasza:

```
{"guest_id": 1, "table_no": 4, "subtotal": 11040.0, "service_charge":
1324.8,
 "discount_percent": 10, "discount_amount": 1236.48, "total": 11128.32}
```

Mentés (5 pont): az API rendelkezik egy mentést segítő végponttal, ahol a mentésre szánt fájl neve (source) és a mentést tartalmazó célfájl (dest) a paraméterek. A mentési és visszatöltési funkciókhoz tartozó fájlok a JSON adatokat kezelő fájlok könyvtárában (data) legyenek elhelyezve. A mentés a Schema alapján valósuljon meg: a beolvasott adatokat a DataStore modellel ellenőrizzük, és csak érvényes adatot írunk ki. Így a lehető legkevesebb adatvesztés vagy adathiba fordulhat elő.

```
@routers.post('/save', status_code=201)
def save(source: str = "data.json", dest: str = "backup.json") -> dict:
    pass
```

Visszatöltés (10 pont): az API rendelkezik egy mentett adatok visszatöltését segítő végponttal. A paraméterei: a visszatöltendő fájl neve (source), a célfájl (dest), a visszatöltés módja (mode), valamint az, hogy összefésüléskor megmaradjanak-e a csak a célfájlban lévő adatok (keep_extra). Sémának nem megfelelő fájl visszatöltése nem ronthatja el a data.json tartalmát!

```
@routers.post('/reload', response_model=ReloadResult)
def reload(source: str = "backup.json", dest: str = "data.json",
           mode: Literal["replace", "merge"] = "replace", keep_extra: bool =
True) -> ReloadResult:
    pass
```

A visszatöltés módjai:

mode	keep_extra	Mi történik
replace (alapértelmezett)	–	A célfájl (data.json) teljes tartalmát lecseréli a mentés tartalmára. A célfájlban lévő, a mentésben nem szereplő adatok elvesznek.
merge	true (alapértelmezett)	A mentésből csak azok az adatok kerülnek vissza, amelyek a célfájlban nincsenek meg. A csak a célfájlban lévő adatok megmaradnak.
merge	false	Mint az előző, de a csak a célfájlban lévő (a mentésben nem szereplő) adatok törlődnek.

Összefésüléskor (merge) az adatokat az azonosítójuk alapján kell párosítani: a vendéget az id, a számlát a vendég (guest_id), a számlán lévő ételt a dish_id alapján. Ha egy rekord mindkét fájlban megvan, a célfájl (data.json) változata marad meg, vagyis a visszatöltés a meglévő adatot nem írja felül. Az összefésülés a számlák ételeire is vonatkozik: egy mindkét fájlban meglévő számlára a mentésből a hiányzó ételek kerülnek fel, keep_extra=false esetén pedig a csak a célfájlban lévő ételek törlődnek.

Az összefésült adatállománynak is ellentmondásmentesnek kell lennie (ezt a DataStore modell szintű validációja ellenőrzi). Ha például a mentésből visszatöltendő számla asztalánál a data.json szerint már egy másik vendég ül, a válasz 409 Conflict, és a data.json nem változik.

A válasz a visszatöltés eredményét is tartalmazza (ReloadResult): hány rekord került be (added), törlődött (removed), illetve változott meg (updated) a visszatöltés előtti állapothoz képest. A számlánál csak az id és a table_no számít (az ételek nem), az ételt a (guest_id, dish_id) pár azonosítja. Példa (mode=merge, keep_extra=true, a mentésben egy új vendég, annak számlája és egy új étel van):

```
{"message": "Sikeres visszatöltés.", "source": "backup.json", "dest":
"data.json",
 "mode": "merge", "keep_extra": true,
 "added":   {"guests": 1, "tabs": 1, "dishes": 1},
 "removed": {"guests": 0, "tabs": 0, "dishes": 0},
 "updated": {"guests": 0, "tabs": 0, "dishes": 0}}
```

Segítség: a mode paraméter értékkészlete a Python beépített typing moduljának `Literal` típusával adható meg. A FastAPI ilyenkor minden más értékre automatikusan 422-es hibát ad, a /docs felületen pedig legördülő listában kínálja fel a lehetséges értékeket. A bool típusú query paramétert a FastAPI a true/false (1/0, yes/no) értékekből alakítja át:

```
from typing import Literal

@routers.get('/pelda')
def pelda(sorrend: Literal["novekvo", "csokkeno"] = "novekvo", reszletes:
bool = False):
    ...
```

Az összefésüléshez és a változások összeszámolásához a filehandler.py `merge_data()` és `diff_counts()` függvényét kell megírni, a pontos szabályok a fájl leírásában találhatók. A rekordok kereséséhez érdemes azonosító szerinti szótárat (dict) vagy halmazt (set) készíteni, pl. `{g["id"]: g for g in data["Guests"]}`; a bemenő adatok módosítása helyett a `copy.deepcopy()` függvénnyel másolatot kell készíteni.

Elvárt viselkedés mindkét végpontnál:

- Sikeres mentés esetén 201, sikeres visszatöltés esetén 200 a státuszkód. A mentés válasza: `{"message": "Sikeres mentés.", "source": "data.json", "dest": "backup.json"}`.
- Csak a data könyvtárban lévő, .json kiterjesztésű fájlnév adható meg. Könyvtárat tartalmazó név (pl. `../main.json`) vagy más kiterjesztés esetén 400 a válasz. Erre a filehandler.py `get_file_path()` függvénye szolgál.
- Nem létező forrásfájl esetén 404, sémának nem megfelelő vagy ellentmondásos tartalom esetén 422, ha a forrás és a cél azonos, akkor 400 a válasz. Visszatöltéskor a nem létező célfájl üres adatállománynak számít; ha az összefésült adatok ellentmondásosak, 409 a válasz, és a célfájl nem változik.

Segítség: a DataStore modell mezői a data.json kulcsai (Guests és Tabs), így a teljes fájl egy lépésben ellenőrizhető:

```
data = load_json_file(source)                       # dict a fájlból
validated = DataStore.model_validate(data)     # ValidationError, ha hibás
save_json_file(validated.model_dump(mode="json"), dest)
```

Tesztelés (5 pont): a végpontokat a beépített tesztelési lehetőségek (/docs) mellett egy külső szoftverrel is teszteljük. Erre alkalmas lehet a Postman alkalmazás, de más szoftver (pl. Insomnia, Thunder Client, curl) is elfogadható. A tesztelésről rövid dokumentációt kell készíteni, amelyet a védés során az alkalmazással együtt be kell

mutatni. A dokumentáció minden végpontnál tartalmazzon legalább egy sikeres és egy hibás kérést (a kérés, a várt és a kapott státuszkód, a válasz törzse).

Az endpoint-ok indokolt esetben módosíthatók, vagy ki is lehet őket egészíteni, de a végpontok neve, HTTP metódusa és paraméterei maradjanak a leírás szerintiek, mert a beadott megoldásokat automatikus teszttel is ellenőrizzük!

Kérdés esetén a gyakorlatvezetőhöz vagy az előadás vezetőjéhez lehet fordulni.

A feladat értékeléséhez minden végpontnak működnie kell. A pontszám a megvalósítás minőségétől is függ, vagyis ha egy végpont működik, de a megadott feltételeknek nem felel meg, akkor kevesebb pontot ér. A tesztelés is követelménye a feladat megoldásának!

A pontszámok összesítése

Részfeladat	**Pont**
addguest végpont	5
opentab végpont	5
orderdish végpont	5
updatedish végpont	10
canceldish végpont	5
guest végpont	5
guests végpont	5
tab végpont	5
getbill végpont	10
Végpont szabályok (response_model, típusok, JSONResponse, hibakezelés, HTTP metódusok és státuszkódok)	10
Az adatok validálása (schema.py)	15
save végpont	5
reload végpont (replace / merge)	10
Tesztelés és dokumentációja	5
Összesen	**100**

Az elérhető maximális pontszám: 100 pont

Jó munkát!